TP 5 : Invocation des méthodes sur des objets distants avec java RMI

Activité 5-2 : Service de banque en utilisant RMI et JNDI

Objectifs

Au terme de cette activité, l'étudiant sera capable de :

- Mettre en œuvre une communication client/serveur via les Remote Method Invocation (RMI) de Java.
- Comprendre l'intégration entre JNDI et RMI pour la recherche d'objets distants.

Environnement

- JDK >= 1.8
- Eclipse IDE for Java developers Ou IntelliJ IDAE > 1.8 OU VS code
- Git, GitHub

Type de l'activité

- Individuel

Ressources

- Consultez le cours « Développement d'applications réparties : RMI) ».

Livrables

- Le lien du repository dans l'espace de dépôt de la plateforme ENT
- Le fichier README

Critères d'évaluation :

- Lien partagé avec un TP conforme aux consignes et dans le délai : **5 pts**,
- Lien partagé avec TP incomplet ou partiellement conforme ou hors délais : **2-4 pts**,
- Absence de dépôt ou hors sujet ou en retard d'une période : **0-1 pt**

Enoncé et travail demandé

Vous allez développer une application répartie qui gère des comptes bancaires.

Un compte bancaire est défini par un code (int), un solde (double), une date de création (Date).

Un Serveur RMI central sera responsable de la gestion de tous les comptes bancaires. Il offrira les services distants suivants aux applications Clientes :

- Créer un Compte avec un code spécifié et une somme initiale (si le code n'existe pas déjà)
- Consulter un Compte à partir de son code.

Cette activité comporte six étapes :

Étape 1 : Préparation de l'espace de travail

Dans le fichier README insérez les captures d'écran de chaque étape réalisée.

Étape 2 : Développement du serveur

1. Créez un nouveau Projet Java nommé **RMIBanque_Server.**
2. Créez un package nommé « **metier** » dans lequel on crée une classe serializable « **Compte** ». Un compte est caractérisé par un code (int), un solde (double) et une
3. date de création (Date).
4. Créez un autre package « **rmiService** » dans lequel on crée l'interface distante « **IBanque** ».

   Cette interface doit définir les méthodes distantes :

   - « **String creerCompte(Compte c)** »
   - « **String getInfoCompte(int code)** »

NB : Importez les packages nécessaires et ajoutez les exceptions appropriées (Remote et RemoteException).

5. Implémentez le service RMI en créant une classe nommée « **BanqueImpl** » dans le même package « rmiService ».
6. Créez un deuxième package nommé « **rmiServer** » dans lequel vous créez une classe appelée « **BanqueServer** » pour le développement de l'application serveur.

Étape 3 : Développement du client

1. Créez un nouveau projet Java nommé « **RMIBanque_Client** ».
2. Exportez les packages « rmiService » et « metier » du projet RMIBanque_Server, en sélectionnant uniquement l'interface IBanque et la classe Compte (ne pas inclure la classe BanqueImpl).
3. Générez un fichier JAR nommé « **banqueClient.jar** » et enregistrez-le dans votre espace de travail.
4. Importez ce fichier JAR dans le projet RMIBanque_Client afin de l'ajouter à son Classpath. Pour cela :

Clic droit sur le projet → Properties → Java Build Path → Libraries → Classpath → Add External JARs... → sélectionner conversionClient.jar → Apply and Close.

5. Créez un package nommé « **rmiClient** », puis ajoutez-y une classe appelée « **BanqueClient** » pour invoquer les méthodes distantes.

Étape 4 : Exécution de l'application

1. Démarrez le serveur RMI.
2. Lancez le client afin de vérifier le fonctionnement de l'application.

Étape 5 : Utilisation de JNDI pour l'Enregistrement RMI

1. Modifiez votre application RMI afin d'utiliser **JNDI** (Java Naming and Directory Interface) pour l'enregistrement (binding) et la recherche (lookup) des objets distants, au lieu d'utiliser directement le registre RMI (LocateRegistry et Naming).
   - **i.** Modifiez la classe **BanqueServer** pour enregistrer l'objet distant BanqueImpl dans un service d'annuaire JNDI.
   - **ii.** Vous devrez configurer l'environnement JNDI en créant le fichier **jndi.properties** situé dans le répertoire src (en y spécifiant **java.naming.factory.initial** et **java.naming.provider.url** pour pointer vers le registre RMI / rmi-registry), puis utiliser l'objet InitialContext pour effectuer le binding de l'objet distant.

Jndi.properties

```
java.naming.factory.initial=com.sun.jndi.rmi.registry.RegistryContextFactory
java.naming.provider.url=rmi://localhost:1099
```

iii. Modifiez la classe **BanqueClient** pour effectuer la recherche (lookup) de l'objet distant IBanque via l'annuaire **JNDI** au lieu d'utiliser Naming.lookup().

iv. De manière similaire au serveur, configurez l'environnement JNDI si nécessaire pour que l'interface InitialContext puisse localiser le service distant.

2. Relancez l'application (Serveur et Client) pour vérifier que la communication RMI fonctionne toujours correctement en utilisant la nouvelle méthode d'enregistrement via JNDI.

Étape 6 : Dépôt des livrables

Dans cette dernière étape, vous êtes invités à :

- Déposez votre projet sur GitHub ainsi que le lien du repository dans l'espace de dépôt de la plateforme ENT.
- Le fichier README doit contenir des imprimés écrans accompagnés d'une synthèse.